Viva Cheat Sheet — One Page

Traffic Congestion Pattern Discovery · NYC Yellow Taxi (6 mo, 2025) · Unsupervised clustering of (zone × hour-of-day) observations.

Aspect	Value
Problem	Discover congestion regimes from taxi-trip telemetry without labels
Unit of observation	(zone × hour-of-day × day-of-week) → ~39,000 rows after aggregation
Spatial domain	260 of 263 official TLC taxi zones (NYC)
Primary algorithm	K-Means (sklearn), n_init=10, random_state=42, max_iter=300
Sanity-check algorithm	DBSCAN (eps=0.5, min_samples=10)
Feature set (5)	hour_sin, hour_cos, is_weekend, speed_deviation, log_trip_density
Scaler	StandardScaler (mean 0, std 1) — Euclidean distance demands it
k selected by	Joint elbow (Kneedle) + silhouette argmax; reconciled within ±1
KPI gates	Silhouette > 0.50 · CV_{intra} < 0.20 · ARI_{min} ≥ 0.70 (5 seeds)
Outputs	model.pkl · scaler.pkl · metrics.json · cluster_labels.parquet · Streamlit app

Top 10 one-liners — memorise these

1. Why unsupervised?	We have telemetry, no ground-truth congestion labels. Clustering discovers structure; supervised learning would require us to invent labels first.
2. Why K-Means over DBSCAN as primary?	We want a fixed, interpretable partition into severity tiers. K-Means gives every zone-hour a label; DBSCAN leaves outliers unassigned, which the dashboard cannot consume.
3. Why *speed_deviation* not raw speed?	Raw speed conflates structural slowness (school zones) with dynamic slowness (congestion). speed_deviation(z, h) = avg_speed(z, h) – baseline(z) isolates the dynamic part.
4. Why drop zone_id?	Zone IDs are nominal categorical. Putting them in a Euclidean metric assumes Zone 23 and Zone 24 are 'closer' than Zone 23 and Zone 100, which has no geometric meaning.
5. Why cyclic time encoding?	Linear hour_of_day puts 23:00 and 00:00 23 units apart; on the unit circle, sin/cos make them adjacent — congestion patterns at midnight should not look maximally different from 11pm.
6. Why log1p(trip_density)?	Trip count is heavy-tailed (Pareto-like). log1p compresses the tail so StandardScaler isn't dominated by airport zones at peak hour.
7. How did you choose k?	Kneedle elbow on inertia + silhouette argmax; if they disagree by >1, prefer silhouette and log the disagreement.
8. How did you validate stability?	Refit K-Means with 5 different random seeds, compute pairwise ARI; min ARI ≥ 0.70 means the partition is reproducible, not an init artefact.
9. Silhouette score interpretation?	Range [–1, 1]. ≈1: tight, well-separated clusters. ≈0: points on the boundary. <0: probably misassigned. We target >0.50.
10. What's a known limitation?	K-Means assumes roughly spherical, equal-variance clusters in Euclidean space. Real traffic regimes can be elongated or unequal in size; that's why we corroborate with DBSCAN.

Formulas you should be able to write on the board

speed_deviation	$\Delta(z, h) = s(z, h) - \text{baseline}(z)$, $\text{baseline}(z) = \text{mean}_h\ s(z, h)$
Cyclic encoding	$\text{hour\_sin} = \sin(2\pi \cdot h / 24)$, $\text{hour\_cos} = \cos(2\pi \cdot h / 24)$
log1p	$\text{log\_trip\_density} = \ln(1 + \text{trip\_density})$; $\text{log1p}(0) = 0$, well-defined at zero

StandardScaler	$x' = (x - \mu) / \sigma$; per-feature, fit on training data
K-Means objective	minimise $J = \Sigma_i \Sigma_{x \in C_i} \|\|x - \mu_i\|\|^2$
Silhouette	$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$; a = avg intra-cluster dist, b = nearest-other-cluster avg dist
Davies–Bouldin	$DB = (1/k) \Sigma_i \max_{j \neq i} [(\sigma_i + \sigma_j) / d(c_i, c_j)]$ — lower is better
Calinski–Harabasz	$CH = [tr(B_k) / tr(W_k)] \cdot [(n - k) / (k - 1)]$ — higher is better
ARI	ARI = (RI – E[RI]) / (max(RI) – E[RI]); +1 perfect, 0 random, <0 worse than random
CV_{intra}	$CV(C_i) = \sigma(\text{speed} \mid C_i) / \mu(\text{speed} \mid C_i)$; homogeneity-of-speed check

Comprehensive Viva Q&A

Eighty questions across motivation, EDA, preprocessing, feature engineering, K-Means mechanics, k-selection, evaluation, stability, DBSCAN, alternatives, limitations, deployment, and PhD-level follow-ups. Read top-to-bottom; the later sections build on earlier ones.

A. Project & Problem Framing

Q. In one sentence, what is the problem this project solves?

A. Given six months of NYC Yellow Taxi trip records and no congestion labels, discover and characterise distinct congestion regimes across NYC's 263 taxi zones at hour-of-day granularity, and expose them through a dashboard.

Q. Why is this an unsupervised problem and not supervised?

A. There is no labelled ground truth for what 'congested' means at zone-hour granularity. Hand-labelling tens of thousands of zone-hour cells is infeasible and would import a researcher's biases. Clustering lets the data tell us how many regimes exist and which observations belong together.

Q. What is the unit of observation, and why?

A. (zone × hour-of-day × day-of-week). One row per (zone, hour, dow) bucket. We picked this granularity because (a) hour resolves rush patterns, (b) day-of-week captures the weekday/weekend regime split, and (c) zone is the smallest stable spatial unit the TLC publishes. Going finer (e.g. minute × lat-lon grid) explodes data sparsity per cell.

Q. Who is the intended end-user and what decision do they make with the output?

A. Operations planners and city analysts. The decision is prioritisation: which (zone, time-window) combinations warrant route re-planning, signal re-timing, or resource allocation. The dashboard surfaces the top-N most congested zone-hours and the temporal pattern.

Q. Why was K-Means chosen as the primary model?

A. Three reasons: (1) every zone-hour must receive a label so downstream maps and KPIs work — DBSCAN's noise label is unusable; (2) the partition must be stable across re-runs for the dashboard to be trustworthy — K-Means with n_init=10 plus seeded ARI ≥ 0.70 satisfies that; (3) interpretability — centroids in a low-D feature space are directly readable as 'fast/slow, busy/quiet' tuples.

B. Dataset & Exploratory Analysis

Q. Describe the raw dataset.

A. NYC TLC Yellow Taxi Trip Records, six months of 2025. Each row is a single trip with pickup/dropoff timestamps, trip distance (mi), pickup zone ID, dropoff zone ID, and optional fare and passenger count. Raw size is ~400 MB across the six monthly parquet files.

Q. From per-trip rows, how do you arrive at the modelling table?

A. Aggregate by groupby(zone_id, hour_of_day, day_of_week, is_weekend) and compute (a) mean speed, (b) trip count = trip_density, (c) mean trip duration. The aggregation collapses ~10M trips down to ~39,000 zone-hour observations, which is the actual input to the cluster model.

Q. What does 'speed' mean per trip when you don't have GPS traces?

A. We compute it analytically: speed_mph = trip_distance / (trip_duration / 3600). It is the average speed over the entire trip, not an instantaneous speed. Limitations: a trip that idles for 5 minutes then sprints will look like a moderate-speed trip; we mitigate by aggregating many trips into each zone-hour cell.

Q. How many zones did you actually have data for, and why not 263?

A. 260 zones. The three missing ones are extremely sparse — typically the outer-fringe zones with negligible yellow-taxi activity. Including them would inject noise from cells with one or two trips.

Q. What did EDA reveal that informed your design choices?

A. Three findings drove the design: (1) raw trip_density spans four orders of magnitude — heavy-tailed and Pareto-like, motivating log1p; (2) raw avg_speed correlates strongly with the kind of street, not with congestion, motivating speed_deviation; (3) hour-of-day shows the canonical bimodal rush pattern on weekdays but a single midday peak on weekends, so is_weekend is informative.

Q. How is the speed_deviation distribution shaped, and what does its shape tell you?

A. Roughly symmetric, centred near zero, std ≈ 2.77 mph, range approximately [–28, +40]. The symmetry is what we want: the negative tail is the congestion signal we are trying to capture, and the fact that mean ≈ 0 means the baseline is well-calibrated. A skewed distribution would have indicated a bug in the baseline computation.

C. Preprocessing — The Cleaning Chain

Q. Walk me through the cleaning pipeline.

A. Twelve named filters applied in order: cast_dtypes → remove_duplicates → remove_nulls → remove_invalid_timestamps → remove_wrong_month → remove_invalid_distance → remove_invalid_duration → remove_speed_outliers → remove_invalid_zones → remove_trivial_same_zone_trips → remove_invalid_quality_signals → optional remove_iqr_speed_outliers_by_zone. Each is one function; each logs how many rows it dropped, which makes the pipeline auditable.

Q. Why are bounds on speed [1, 100] mph and not [0, ∞]?

A. Below 1 mph for a logged trip is almost certainly a meter-on/meter-off non-trip or a stuck GPS. Above 100 mph in NYC traffic is physically implausible — it indicates a corrupted timestamp. The bounds protect the baseline computation from outlier-driven distortion.

Q. What is the IQR fence and why is it 'opt-in'?

A. For each zone with ≥ 200 trips, compute Q1, Q3 of speed and define [Q1 – 3·IQR, Q3 + 3·IQR] as the per-zone fence; drop trips outside. It is opt-in because the global cleaning chain is already conservative; the IQR fence is for a stricter pass when the downstream model is sensitive to outliers. K-Means is moderately sensitive to outliers (squared distance), so for production we ran with it on.

Q. Why drop trips with PU == DO and distance ≤ 0.10 mi?

A. Those are almost always meter-on, meter-off events (driver started the meter and then realised the customer cancelled, or moved a few feet). Including them inflates trip_density without contributing real movement information.

Q. How do you handle the timezone / month-boundary edge case?

A. remove_wrong_month parses YYYY-MM from the filename and drops any pickup outside that month. Month-end stragglers (a midnight Dec 31 pickup ending Jan 1) are bucketed by pickup time, so they stay if they belong to the expected month. We do not localise across timezones because all NYC data is already in EST/EDT.

Q. What's the retention rate after cleaning, and is that healthy?

A. Roughly 88–92% of rows survive in a typical month (you'd quote the exact percentage from your run logs). Healthy in this domain — TLC data is well-curated upstream, so most drops are timestamp inversions and near-zero same-zone trips, which are clear data-quality artefacts.

D. Feature Engineering — The Core of the Project

Q. Why is feature engineering the most important methodological step here?

A. K-Means uses raw Euclidean distance on whatever you hand it. A bad feature set guarantees a bad partition no matter how clever the algorithm. speed_deviation alone determines whether the model can distinguish structural slowness (a 20 mph residential street) from dynamic slowness (Times Square at 5

PM). That single design choice is load-bearing.

Q. Why exactly five features? Could fewer or more be better?

A. Three principles guided the choice: (a) every feature must be interpretable as a 'why this cell is congested' signal, (b) features must be on commensurate Euclidean scales after StandardScaler, (c) features should not be nearly-collinear. Adding raw avg_speed_mph would duplicate speed_deviation; adding raw hour_of_day would defeat the cyclic encoding; adding zone_id would inject categorical noise. Five is the minimal spanning set.

Q. Walk me through the speed_deviation formula on the board.

A. For every (zone, hour-of-day) pair: $\Delta(z, h) = s(z, h) - \text{baseline}(z)$, where baseline(z) is the mean speed across all hours within zone z. Implemented in one groupby: agg.groupby(['zone_id', 'hour_of_day'])['avg_speed_mph'].mean() gives the baseline; subtract from each row. A positive Δ means 'this cell is moving faster than this zone normally does'; a negative Δ means 'this cell is congested relative to this zone's norm'.

Q. Why subtract a per-zone baseline rather than a global baseline?

A. Manhattan averages ~10 mph; the BQE averages ~30 mph. Subtracting a global baseline labels half of Manhattan as 'always congested' and half of the highways as 'always uncongested' — that is geography, not congestion. Per-zone baselining centres each zone on its own normal so the deviation signal is comparable across the city.

Q. Defend cyclic time encoding against just using hour_of_day directly.

A. K-Means uses Euclidean distance. With raw hour_of_day, points at h=23 and h=00 are 23 units apart — maximally distant — even though they are consecutive. Encoding hour as $(\sin(2\pi h/24), \cos(2\pi h/24))$ puts hours on the unit circle, where 23:00 and 00:00 are adjacent. The two new features are bounded in [–1, 1], naturally Euclidean-friendly, and inject zero ordering bias.

Q. Why log1p and not log? And not Box-Cox?

A. $\text{log1p}(x) = \ln(1 + x)$. It is well-defined at $x = 0$ (trips can be zero in a cell), gives the same heavy-tail compression as log on large values, and is fully invertible (expm1). Box-Cox needs strictly positive inputs and estimates a λ parameter — extra moving part for marginal benefit on a feature whose distribution is well-modelled by a simple log transform.

Q. Why is is_weekend kept as binary instead of one-hot encoding day_of_week?

A. We tested one-hot: it added six features that were largely redundant with is_weekend and inflated the feature-space dimensionality, hurting silhouette. Empirically the weekday/weekend split is the dominant temporal regime; finer day-of-week resolution does not change the cluster structure enough to justify the extra dimensions.

Q. Why is zone_id excluded from the cluster features but kept in the dataframe?

A. It is a nominal categorical with 263 levels; embedding it Euclidean-wise would assert that zone 23 is closer to zone 24 than to zone 100, which has no geometric meaning. We keep it as a join key so we can attach the zone label back to each clustered row for downstream visualisation.

Q. Why didn't you include weather, accidents, or events as features?

A. Two reasons: (a) data availability — NYC weather and 311 incident data exist but are noisy and sparsely aligned to zone-hour; (b) scope discipline — the project's hypothesis is that congestion regimes are detectable from trip telemetry alone. Adding exogenous covariates would conflate the discovery question ('what regimes exist?') with an attribution question ('what causes them?'). Future work could add them.

Q. Could you have used embedding-based features (e.g. autoencoder over raw trips)?

A. Yes — a deep autoencoder on the raw trip stream could in principle learn a lower-dimensional manifold from which to cluster. We deliberately chose interpretable hand-crafted features for two reasons: (a) viva and stakeholder defensibility — every centroid coordinate has a name and a unit; (b) data efficiency — autoencoders need much more data and tuning than 39k zone-hour rows. A scaled-up production version could compare both.

E. Methodology — Why Unsupervised, Why K-Means

Q. State the formal definition of unsupervised learning.

A. Given an unlabelled dataset $\{x_i\} \subset R^d$, find structure — clusters, density modes, low-dimensional manifolds, or generative models — without target variables y_i. The objective is intrinsic, defined on the data alone, not on prediction error against labels.

Q. Why is K-Means appropriate for THIS problem specifically?

A. Three boxes ticked: (a) we want a partition (every zone-hour gets a label), K-Means partitions exhaustively; (b) the latent regimes are plausibly convex blobs in the engineered feature space (low-traffic-fast, high-traffic-slow, etc.), which matches K-Means' inductive bias; (c) computational cost — K-Means is O(n·k·d·iter) and trivially fits 39k × 5 in milliseconds, so we can sweep k cheaply and run the multi-seed stability battery.

Q. State the K-Means objective function precisely.

A. Minimise the within-cluster sum of squares (WCSS): $J(C, \mu) = \Sigma_{i=1..k} \Sigma_{x \in C_i} ||x - \mu_i||^2$, where C_i is the i-th cluster and μ_i is its centroid. K-Means is a coordinate-descent heuristic — it does not guarantee a global optimum.

Q. How does Lloyd's algorithm work, step by step?

A. (1) Initialise k centroids (we use k-means++ via sklearn's default). (2) Assignment step: assign each point to its nearest centroid. (3) Update step: move each centroid to the mean of its assigned points. (4) Repeat 2–3 until assignments stop changing or max_iter is hit. Each iteration monotonically decreases J, but the final J depends on initialisation — hence n_init = 10 (run from 10 random seeds, keep best).

Q. Why k-means++ over uniform random initialisation?

A. Uniform random can place all initial centroids in one dense region, leading to bad local minima and slow convergence. k-means++ samples the first centroid uniformly, then samples each subsequent centroid with probability proportional to $D(x)^2$ where D(x) is the distance to the nearest already-chosen centroid. This spreads the seeds and gives an expected O(log k) approximation guarantee on the optimum.

Q. Could you have used hierarchical clustering instead?

A. Yes for a smaller dataset; not for this one. Agglomerative clustering is $O(n^2 \log n)$ in time and $O(n^2)$ in memory; 39k points means a 1.5-billion-entry distance matrix — unworkable. We could subsample, but then we would lose the very rare congestion events that are the most interesting cells.

F. K-Means — Math & Mechanics

Q. What is 'inertia' in sklearn's KMeans?

A. Inertia is the fitted value of the K-Means objective J — the sum of squared Euclidean distances from each point to its assigned centroid. Lower is better, and it monotonically decreases as k grows (k = n gives inertia 0). That is why we cannot pick k by minimising inertia alone.

Q. What does 'random_state' actually control?

A. Two stochastic steps: (a) the seed for k-means++ centroid sampling, and (b) the order in which sklearn evaluates the n_init replicates. With a fixed random_state, the entire fit is bitwise reproducible. We use random_state=42 for the production fit and seeds 0..4 for the stability battery.

Q. Why n_init = 10?

A. K-Means is a non-convex problem; a single Lloyd run can land in a poor local minimum depending on initialisation. n_init = 10 runs Lloyd ten times with different k-means++ seeds and keeps the best (lowest-inertia) result. It is a cheap defence against bad starts and is sklearn's default.

Q. What does max_iter = 300 protect against?

A. Pathological cases where Lloyd's iterations oscillate between near-equal cluster assignments without strictly converging. In practice K-Means on this dataset converges in ~10–30 iterations; 300 is a safety

ceiling. n_iter_ on the fitted model tells you how many were actually used.

Q. Why does K-Means assume Euclidean distance?

A. The objective minimises squared Euclidean distance. The update step (μ_i = mean of cluster) is the analytical minimiser of that objective only under Euclidean geometry. With other metrics (e.g. Manhattan), you'd need k-medoids; with cosine similarity, a different objective entirely.

Q. What happens if your features are correlated?

A. Correlated features double-count the same direction in feature space, biasing the partition toward that direction. We checked: hour_sin and hour_cos are by construction uncorrelated; speed_deviation and log_trip_density have $\rho \approx -0.4$ (busier $\leftrightarrow$ slower, expected and not redundant); is_weekend is roughly orthogonal. Mahalanobis distance would decorrelate, but at the cost of needing a covariance estimate from the data — extra moving part with marginal gain.

Q. Why is K-Means sensitive to feature scale?

A. Euclidean distance is dominated by the feature with the largest variance. Without scaling, log_trip_density (range ~0–10) would swamp is_weekend (range 0–1), and the partition would be driven almost entirely by traffic volume. StandardScaler equalises the contribution: every feature has unit variance after scaling.

Q. Walk me through the convergence guarantee for K-Means.

A. The objective J is bounded below by 0. The assignment step (each point to nearest centroid) cannot increase J. The update step (centroid to cluster mean) cannot increase J either, because the cluster mean is exactly the J-minimiser given fixed assignments. So J is monotonically non-increasing and bounded — by monotone convergence, the sequence converges. It does not converge to a global optimum, just a fixed point.

G. Scaling & Distance Geometry

Q. StandardScaler vs RobustScaler — when would you choose which?

A. StandardScaler subtracts the mean and divides by standard deviation: $x' = (x - \mu)/\sigma$. Sensitive to outliers because both μ and σ are. RobustScaler uses the median and IQR: $x' = (x - Q_2)/\text{IQR}$. Robust to outliers. We chose StandardScaler because the cleaning chain already aggressively removes outliers; given clean data, the parametric scaler gives a tighter unit-variance result and slightly better silhouette.

Q. Are your features normally distributed after scaling?

A. Not exactly. StandardScaler standardises mean and variance but does not normalise the shape. speed_deviation is roughly Gaussian post-scaling; is_weekend remains a discrete bimodal at two points; hour_sin/cos are uniform on [–1, 1] then rescaled. Normality is not required by K-Means — only commensurate scales.

Q. What is the curse of dimensionality and does it apply here?

A. In high dimensions, the ratio of nearest-to-farthest neighbour distance approaches 1, so distance-based methods (including K-Means) degenerate. Empirically the curse bites hard around d ■ 50–100. We have d = 5, comfortably below the regime where the curse is problematic. Distance discrimination is preserved.

Q. Why not use a kernel method like spectral clustering?

A. Spectral clustering builds an n×n similarity graph, computes the graph Laplacian, takes its leading eigenvectors, and clusters there. Beautiful but expensive ($O(n^3)$ for full eigendecomposition; approximations exist but add hyperparameters). For 39k points and 5 interpretable features, the marginal gain over K-Means does not justify the loss of interpretability — centroids in our space are directly human-readable; spectral cluster centres in eigenspace are not.

Q. If you had to pick a different distance metric, which and why?

A. Mahalanobis distance, if I wanted to factor out feature correlations automatically. Cosine similarity would be the wrong choice — our features are not directional in a meaningful sense; magnitude matters.

H. Choosing the Number of Clusters

Q. How did you choose k? Walk me through the procedure.

A. Sweep k = 2..10. For each k: fit K-Means, record inertia and silhouette (silhouette is computed on a 50,000-point sample for speed). Then: (1) detect the elbow on the inertia curve using the Kneedle algorithm (2nd-derivative argmax fallback if Kneedle is unavailable); (2) find the silhouette argmax. If the two agree within ±1, pick the elbow; otherwise prefer silhouette and log the disagreement so a human can override.

Q. Why both elbow and silhouette? Aren't they redundant?

A. No — they answer different questions. The elbow asks 'where does adding a cluster stop meaningfully reducing within-cluster scatter?'. Silhouette asks 'at this k, are the clusters well-separated and compact in absolute terms?'. Elbow can be ambiguous on smooth inertia curves; silhouette is a more discriminating quality metric but can be tricked by tight singleton clusters. Combining them gives a more defensible choice.

Q. Explain the Kneedle algorithm in one paragraph.

A. Kneedle detects the 'knee' of a curve by computing the perpendicular distance from each point to the chord connecting the curve's endpoints, after normalising x and y to [0, 1]. The point with maximum perpendicular distance is the knee. It works robustly on monotonically decreasing-and-concave curves like K-Means inertia.

Q. What is silhouette intuitively, and what is its formal definition?

A. Intuitively: 'how much closer is this point to its own cluster than to the next-nearest cluster?'. Formally for point i: a(i) = mean distance to other points in its cluster; b(i) = min over other clusters C of the mean distance from i to points in C. Silhouette $s(i) = (b(i) - a(i)) / \max(a(i), b(i)) \in [-1, 1]$. The score is the mean of s(i) over all points.

Q. What's a 'good' silhouette score? Are the textbook thresholds reliable?

A. Rough conventions: > 0.7 strong, 0.5–0.7 reasonable, 0.25–0.5 weak, < 0.25 essentially no structure. These are heuristics, not gospel. They depend on dimensionality and shape — silhouette tends to be lower in higher dimensions even for genuinely good partitions. Always interpret the score alongside DB, CH, and visualisation.

Q. Why do you sample 50,000 points for silhouette?

A. Silhouette is $O(n^2)$ because it computes pairwise distances. On 39k points, the full computation is fast; on 1M+ rows it would be prohibitive. We use a fixed-seed sample so the score is reproducible. Sample-based silhouette is an unbiased estimator with variance shrinking as 1/sqrt(sample_size).

Q. If your elbow and silhouette disagreed by 3 or more, what would you do?

A. First, distrust both numbers. Second, run the multi-seed ARI battery at each candidate k and prefer the one with the highest stability. Third, interpret the centroids — does each cluster have a coherent operational meaning? A choice that scores middlingly on silhouette but yields five interpretable, stable, well-sized clusters is operationally superior to a choice that scores best on silhouette but produces a degenerate partition.

I. Evaluation Metrics — Internal & External

Q. Distinguish internal, external, and relative cluster validation.

A. Internal validation uses only the data and the partition (silhouette, DB, CH, inertia). External validation compares to ground-truth labels (ARI, NMI, V-measure) — we don't have those. Relative validation compares two partitions to each other (ARI between seeds, our stability metric).

Q. Define Davies–Bouldin index and explain why lower is better.

A. $DB = (1/k) \Sigma_i \max_{j \neq i} [(\sigma_i + \sigma_j) / d(c_i, c_j)]$, where σ_i is the average distance from cluster-i points to its centroid c_i, and d is centroid-to-centroid distance. The ratio is high when within-cluster scatter is large or between-cluster distance is small. DB averages the worst-case ratio per cluster, so smaller is better. < 1.0 is

a typical 'good' threshold.

Q. Define Calinski–Harabasz index.

A. CH = $[\mathrm{tr}(B_k) / \mathrm{tr}(W_k)] \cdot [(n - k) / (k - 1)]$, where W_k is the within-cluster scatter matrix and B_k is the between-cluster scatter matrix. It rewards partitions where between-cluster variance is large relative to within-cluster variance, with a degrees-of-freedom correction. Higher is better. Unbounded above, so absolute values are not directly comparable across datasets.

Q. What is the intra-class coefficient of variation (CV_{intra}) and why measure it?

A. For each cluster, $CV(C_i) = \sigma(\text{speed} \mid C_i) / \mu(\text{speed} \mid C_i)$. It measures how homogeneous each cluster is in terms of avg speed. The PRD targets $CV_{intra} < 0.20$ for every cluster — meaning within any one cluster, the standard deviation of speed is at most 20% of the mean speed. This is a stronger constraint than silhouette (which is geometric); it directly says 'congestion regimes must be tight in their headline variable'.

Q. What is the Adjusted Rand Index and how is it computed?

A. RI counts agreements between two partitions over all pairs of points: how many pairs are co-clustered in both, or separately-clustered in both. ARI = (RI − E[RI]) / (max(RI) − E[RI]) corrects for chance agreement. ARI = 1 means perfect agreement; ARI = 0 is random; negative is worse-than-random. We compute pairwise ARI across 5 seed-fits as a stability metric.

Q. Why is the silhouette per cluster informative beyond the overall silhouette?

A. The overall silhouette is a mean — it can hide a degenerate cluster. If four clusters have silhouette 0.65 and a fifth has 0.05, the overall average looks fine but cluster 5 is a 'junk drawer'. Per-cluster silhouette flags exactly which cluster is problematic, enabling targeted investigation.

Q. How do you decide if a cluster is 'real' versus an artefact?

A. Three checks: (1) cluster sizes — is any cluster < 1% of the data? It may be capturing outliers, not a regime. (2) Per-cluster silhouette — is any cluster substantially below the overall? Likely a junk drawer. (3) Centroid interpretation — does each cluster's centroid translate into a coherent operational story? If a cluster centroid says 'fast on weekdays, slow on weekends, low density' that's a real regime; if it's a noisy mixture, retrain with k-1.

Q. Why is inertia not a sufficient evaluation metric?

A. Inertia monotonically decreases with k; setting k = n gives inertia 0. It cannot tell you whether the clusters are well-separated or compact in an absolute sense — it only measures within-cluster scatter. Silhouette, DB, and CH all incorporate between-cluster distance, which is what we actually care about.

Q. How do these metrics behave under sample reweighting or imbalanced clusters?

A. Silhouette and DB are sensitive to imbalance: a tiny cluster surrounded by a large one can score badly even when the partition is meaningful. CH is more robust because it's variance-based and incorporates n − k. ARI is invariant to label permutation but not to cluster size imbalance. Always check the cluster-size distribution alongside the metrics.

Q. What's the difference between silhouette and the Dunn index?

A. Both measure 'separation vs cohesion'. Silhouette is averaged over points and bounded in [–1, 1]; Dunn is a ratio (min inter-cluster distance / max intra-cluster diameter), unbounded above, more sensitive to outliers because of the max in the denominator. Silhouette is more widely used and what sklearn provides; Dunn is informative when worst-case behaviour matters more than average.

J. Stability, Reproducibility, and Generalisation

Q. Why is stability analysis necessary for a clustering project?

A. K-Means is a non-convex, randomised heuristic. Two runs from different seeds can land on different local minima with very different cluster labels. If our partition is sensitive to the seed, the labels we ship to the dashboard are an artefact, not a regime. We need to demonstrate that across many seeds the algorithm finds the same structure.

Q. How exactly do you measure stability?

A. Refit K-Means with seeds 0..4, keeping all other hyperparameters fixed. Compute pairwise ARI for all C(5, 2) = 10 pairs of fits. Report min and mean ARI. Min ARI ≥ 0.70 is our gate; mean ≥ 0.90 is a stronger indicator. Five seeds is a balance — more would be more reliable but linearly more expensive.

Q. What does it mean if your min ARI is below threshold?

A. Two possibilities: (a) the chosen k is too high — K-Means is splitting a single regime arbitrarily, and the split varies by seed; or (b) the feature space is too noisy — adjacent regimes overlap enough that boundary points jump clusters. Diagnosis: rerun the elbow + silhouette sweep, decrement k by 1, re-test stability.

Q. How would you generalise this analysis to new months of data?

A. Persist scaler.pkl and model.pkl. For new data: apply the same cleaning chain → engineer the same features → scaler.transform → kmeans.predict(). Critically: do NOT re-fit on each month, or labels will drift. Drift detection: compute the cluster-size distribution and silhouette on new data and alert if either shifts beyond a threshold (e.g. KL divergence > 0.1).

K. DBSCAN — The Independent Sanity Check

Q. Why DBSCAN as a sanity check, given K-Means is the production model?

A. DBSCAN belongs to a different family — density-based, not centroid-based. It makes no assumption that clusters are convex or that every point must be assigned. If DBSCAN's cluster structure broadly aligns with K-Means' (in number and rough composition), it corroborates the partition. If DBSCAN finds one giant cluster plus mostly noise, K-Means is forcing structure on a unimodal density.

Q. Explain DBSCAN's hyperparameters.

A. eps (ε) is the neighbourhood radius — two points are 'close' if their distance ≤ ε. min_samples is the minimum number of neighbours required for a point to be a 'core point'. We use eps=0.5 (in scaled space, so it's half a standard deviation) and min_samples=10. Points that aren't core and aren't density-reachable from any core are labelled noise (-1).

Q. How do you set eps?

A. k-distance plot: for each point, compute distance to its k-th nearest neighbour (k = min_samples). Sort descending. The 'knee' of that curve is a principled eps. We chose 0.5 by both the knee and a domain-knowledge sanity check (in 5-D scaled space, half a std-dev should connect operationally similar zone-hours).

Q. What does it mean when DBSCAN flags 30%+ of points as noise?

A. Either (a) eps is too small or min_samples too large — the density threshold is stricter than the data supports; or (b) the data genuinely is mostly noise/outliers in feature space — there are no dense modes to cluster around. We log a warning when noise fraction exceeds 30% so a human can investigate. Below that threshold, DBSCAN's signal is meaningful.

Q. Could you have used DBSCAN as the primary model?

A. No, three strikes: (a) DBSCAN leaves outlier zone-hours unlabelled, but the dashboard requires every cell to have a label; (b) DBSCAN's number of clusters is implicit and varies with eps — you cannot guarantee a fixed k for stable downstream colour scales; (c) tuning eps is a global hyperparameter — but density varies across feature space, so a single eps may over-merge in dense regions and over-split in sparse ones. HDBSCAN addresses (c); but (a) and (b) still hold.

Q. What's the time complexity of DBSCAN?

A. O(n log n) average case with a spatial index (ball-tree or KD-tree); $O(n^2)$ worst case. Sklearn uses ball-tree by default. For 39k × 5 it runs in seconds.

L. Alternative Algorithms — What You Could Have Used

Q. Compare K-Means and Gaussian Mixture Models.

A. GMMs assume each cluster is a multivariate Gaussian; K-Means assumes spherical clusters of equal variance. GMM gives soft (probabilistic) assignments; K-Means gives hard assignments. GMM is a special case of K-Means' generalisation: K-Means ≈ GMM with isotropic equal-variance Gaussians and hard EM. GMM costs more computationally and has more parameters (covariance matrices) — useful when clusters are elongated or have unequal variance. We tested GMM; silhouette was marginally lower and stability was worse, so we kept K-Means.

Q. When would HDBSCAN be a better choice than K-Means?

A. When clusters have widely varying density and you don't know k in advance. HDBSCAN performs hierarchical density-based clustering and extracts a flat clustering by 'cluster excess of mass' — no global eps. Useful for exploratory analysis where regime sizes are unknown. Trade-off: still leaves noise points unassigned.

Q. Explain agglomerative hierarchical clustering and its drawbacks here.

A. Start with every point as its own cluster; repeatedly merge the two closest clusters until k remain. Linkage criteria (single, complete, average, Ward) determine 'closest'. Drawbacks here: $O(n^2)$ memory for the distance matrix is prohibitive at 39k points; merges are irrevocable, so an early bad merge propagates.

Q. What is spectral clustering, briefly?

A. Build a similarity graph (e.g. k-NN graph or Gaussian-weighted), compute the graph Laplacian L = D − W, take its bottom k eigenvectors as a new embedding, and run K-Means in that embedding. It can capture non-convex cluster shapes that K-Means cannot. Cost: eigendecomposition of an n×n matrix — expensive without approximations.

Q. Briefly: BIRCH, OPTICS, Mean-shift.

A. BIRCH builds a CF-tree for streaming/very-large data — incremental, but less accurate. OPTICS is an ordering-based DBSCAN variant that handles varying density (HDBSCAN is its modern descendant). Mean-shift finds modes by gradient ascent on a kernel density estimate — bandwidth-sensitive and doesn't scale well.

Q. Could a self-supervised representation (e.g. SimCLR) help?

A. Conceivably, with much more data and a learning task that captures operational similarity. The challenge is defining the contrastive objective: what makes two zone-hour observations 'similar' in a way that training on it produces a useful embedding for clustering? Without that, you risk learning a representation that improves a proxy task but not the actual congestion-regime separation. Out of scope here; a candidate for a master's thesis.

M. Limitations & Honest Caveats

Q. What are the main limitations of this work?

A. (1) Yellow Taxi only — no Uber/Lyft, no private vehicles, no buses; the regimes we identify are taxi-visible regimes. (2) K-Means' isotropic-cluster assumption is a simplification of real regimes which may be elongated or asymmetric. (3) Static partitioning — the model assumes regimes are stable across the six months; concept drift between months is not modelled. (4) No causal claim — clustering tells you 'what regimes exist', not 'why this zone is congested'. (5) Spatial correlations between adjacent zones are not modelled; each zone is treated independently.

Q. Where does K-Means fail on this kind of data?

A. Three known failure modes: (a) zones with bimodal speed distributions (e.g. a highway segment that alternates between 5 mph and 50 mph) get mean-collapsed in aggregation, hiding the bimodality before clustering; (b) very rare regimes (a major event) get absorbed into the nearest cluster rather than forming their own; (c) anisotropic regimes where speed_deviation and log_trip_density vary at very different scales — we mitigated with StandardScaler, but residual anisotropy remains.

Q. How would you address the static-partition limitation?

A. Two approaches: (a) sliding-window clustering — fit on the most recent N weeks, compare partitions, alert on drift; (b) online clustering — use Mini-Batch K-Means with a forgetting factor, so the model adapts incrementally. Both add operational complexity; the right choice depends on how fast regimes drift in practice.

Q. Is your evaluation methodology susceptible to data leakage?

A. There is no train/test split because clustering is unsupervised — every row is fed to the algorithm. This means our silhouette/ARI scores are in-sample. The honest framing: stability across seeds is a proxy for generalisation. A stronger guarantee would be a temporal hold-out: fit on months 1–4, score the partition's cluster assignments on months 5–6, and check that silhouette doesn't degrade. Future work.

Q. What's the worst-case operational risk if a downstream user trusts your model blindly?

A. Acting on a degenerate or unstable partition. Concrete scenario: a city operations team re-times traffic signals based on cluster-3 ('high congestion'). If our cluster definitions drift across runs (low ARI), they re-time signals for what was cluster 3 last week and cluster 5 this week — opposite policies. Mitigations: pin model.pkl in production, version every fit, gate deployments on stability metrics.

N. Deployment — The Streamlit Dashboard

Q. How is the model served to end-users?

A. A Streamlit web application (app/app.py) with three companion pages: How It Works, Model Diagnostics, Cluster Stories. The app loads cluster_labels.parquet, model.pkl, scaler.pkl, and metrics.json from models/ and outputs/. The map view uses pydeck (deck.gl) to render zones coloured by their dominant cluster at the user-selected hour.

Q. Why a dashboard instead of an API?

A. Audience. The intended consumers are operations planners and analysts, not machines. They want to slice by hour, day-type, and cluster, see the city map shade in real time, and read narrative cluster summaries. An API serves a different need (programmatic integration); both could coexist, but the dashboard is the primary deliverable.

Q. How is reproducibility guaranteed across the train→serve boundary?

A. Serialise scaler and model: scaler.pkl (StandardScaler with fitted μ, σ) and model.pkl (KMeans with fitted centroids). The serving code does scaler.transform → model.predict, never re-fits. metrics.json records the exact k, feature list, KPI values, and timestamp of the run, so any partition rendered in the dashboard is traceable to a specific fit.

O. PhD-Level & Tricky Follow-ups

Q. Your silhouette is 0.55 and your DB is 0.85. Could those be inconsistent?

A. They can be in tension on small or imbalanced clusters but here both agree the partition is reasonable. Silhouette of 0.55 says boundary points are still clearly closer to their own cluster than to the next; DB of 0.85 says the worst cluster pair has scatter-to-distance ratio below 1, which is the rough 'tight' threshold. Genuine inconsistency — high silhouette, high DB — would suggest one cluster is unusually scattered while the rest are tight; check per-cluster silhouette to confirm.

Q. How would you bootstrap a confidence interval on the silhouette score?

A. Resample 39k zone-hours with replacement, B = 1000 times. For each bootstrap sample, recompute the silhouette using the existing fitted centroids (do NOT re-fit). Take the 2.5th and 97.5th percentiles. Caveat: this measures sampling uncertainty in the silhouette estimate, not in the model itself — model uncertainty is captured by the seed-stability ARI.

Q. Suppose your reviewer says 'why not just normalise to [0, 1] with MinMaxScaler?'

A. MinMaxScaler maps [min, max] → [0, 1]. Sensitive to outliers because min and max are. After our cleaning chain, outliers are mostly removed, so MinMaxScaler would behave reasonably — but the scaled

features would no longer have unit variance. Two features with the same [0, 1] range but very different distributions (e.g. is_weekend at exactly {0, 1} vs. speed_deviation continuous in [0, 1]) would contribute very different amounts to Euclidean distance. StandardScaler equalises that.

Q. Why might your clusters look great on this dataset but fail to generalise?

A. Sample selectivity. Yellow Taxi data is biased toward Manhattan and the airport corridors — outer-borough zones are sparsely covered. The clusters we discover may capture taxi-visible regimes well but miss regimes that exist in Brooklyn or Queens where taxis rarely go. The fix is to combine modalities (TLC for-hire, Citi Bike, MTA bus AVL) — future work.

Q. How is your project related to the literature on unsupervised anomaly detection in mobility data?

A. Adjacent but distinct. Anomaly detection asks 'which observations are unusual relative to the bulk?' — typically uses one-class SVM, isolation forest, or autoencoder reconstruction error. Our project asks 'how many regimes does the bulk decompose into?' — the answer is a partition where every observation belongs somewhere, and 'severe congestion' is a regime, not an anomaly. The two viewpoints complement each other; you could pipeline them by clustering first, then running anomaly detection within each cluster to find 'unusual' zone-hours within their regime.

Closing — How to Carry Yourself in the Viva

Be honest about limitations. Examiners respect a candidate who can say 'this is what we did NOT do, and here's why' more than one who claims the model is universally correct. Every honest caveat in section M above is a defensive moat against being trapped.

Defend every choice with a reason. Not 'I read it in a tutorial' but 'we chose StandardScaler over MinMaxScaler because Euclidean distance demands commensurate variance, and the cleaning chain already removes the outliers MinMax would otherwise be sensitive to.' Three-clause defences.

Know your numbers. Memorise: k chosen, silhouette score, min ARI, cluster sizes, dataset size, number of features. If you fumble a number, the examiner will probe further; if you nail them, they will move on.

If you don't know, say so — and reason aloud. 'I haven't profiled that exactly, but here's how I'd approach it: …'. This shows methodology even when factual recall fails.

Keep your domain story tight. One paragraph: who uses this, what decision they make, why your model improves that decision. Examiners often start there — answer it crisply and the rest of the viva goes downhill from a position of confidence.